

# AI휴먼 | 6일차 | Python 함수(Function)

과정명

AI휴먼

강의일차

6일차

강의주제

Python 함수(Function)

핵심 키워드

def, Parameter, Argument, return, Scope

오늘의 결과물

함수형 성적 처리 프로그램

## 발표 포인트

5일차에는 반복문과 리스트·딕셔너리로 여러 데이터를 처리했습니다.

6일차에는 반복되는 처리 로직을 함수로 분리하여 재사용 가능한 코드로 바꿉니다.

# 오늘의 학습 목표

- 1 함수가 필요한 이유를 설명할 수 있습니다.
- 2 `def` 문으로 함수를 정의하고 호출할 수 있습니다.
- 3 매개변수(Parameter)와 인수(Argument)의 차이를 구분할 수 있습니다.
- 4 `return`으로 처리 결과를 함수 밖으로 전달할 수 있습니다.
- 5 지역변수와 전역변수의 차이를 이해할 수 있습니다.
- 6 조건문·반복문·리스트·딕셔너리를 함수와 결합할 수 있습니다.
- 7 하나의 프로그램을 여러 함수로 나누어 설계할 수 있습니다.

# 5일차 내용 연결: 반복문과 컬렉션

## 이전 예제

```
scores = [72, 88, 95, 61]
for score in scores:
    if score >= 80:
        print(score, "PASS")
    else:
        print(score, "RETRY")
```

## 문제점

같은 판정 규칙이 여러 곳에서 필요하면  
코드가 반복됩니다.

판정 기준을 바꾸면 여러 위치를  
함께 수정해야 합니다.

코드가 길어질수록 역할을 구분하기  
어려워집니다.

## 오늘의 개선 방향

"점수 판정"이라는 하나의 작업을 함수로 분리합니다.

# 함수(Function)란?

## 함수(Function)

특정 작업을 수행하도록 묶어 놓은 재사용 가능한 코드 블록입니다.

입력을 받을 수 있고, 처리 후 결과를 반환할 수 있습니다.

## 일상생활 비유 — 커피 머신

- 입력: 원두, 물, 메뉴 선택
- 처리: 분쇄, 추출
- 출력: 커피

## 프로그램의 함수

- 입력: 매개변수(Parameter)
- 처리: 함수 본문(Body)
- 출력: 반환값(Return Value)

☐ 핵심 구조: 입력 → 함수 내부 처리 → 결과 반환



# 함수를 사용하는 이유



## 재사용성(Reusability)

한 번 작성한 코드를 여러 번 호출할 수 있습니다.



## 가독성(Readability)

코드의 역할을 함수 이름으로 표현할 수 있습니다.



## 유지보수성(Maintainability)

규칙이 바뀌면 함수 한 곳을 수정하면 됩니다.



## 테스트 용이성(Testability)

작은 함수 단위로 결과를 확인할 수 있습니다.



## 역할 분리(Decomposition)

큰 문제를 작은 작업으로 나눌 수 있습니다.

## 예

- `get_grade()`: 등급 판정
- `get_average()`: 평균 계산
- `print_report()`: 결과 출력

# 함수 정의와 호출

## 기본 문법

```
def 함수이름():  
    실행문  
  
함수이름()
```

## 예제

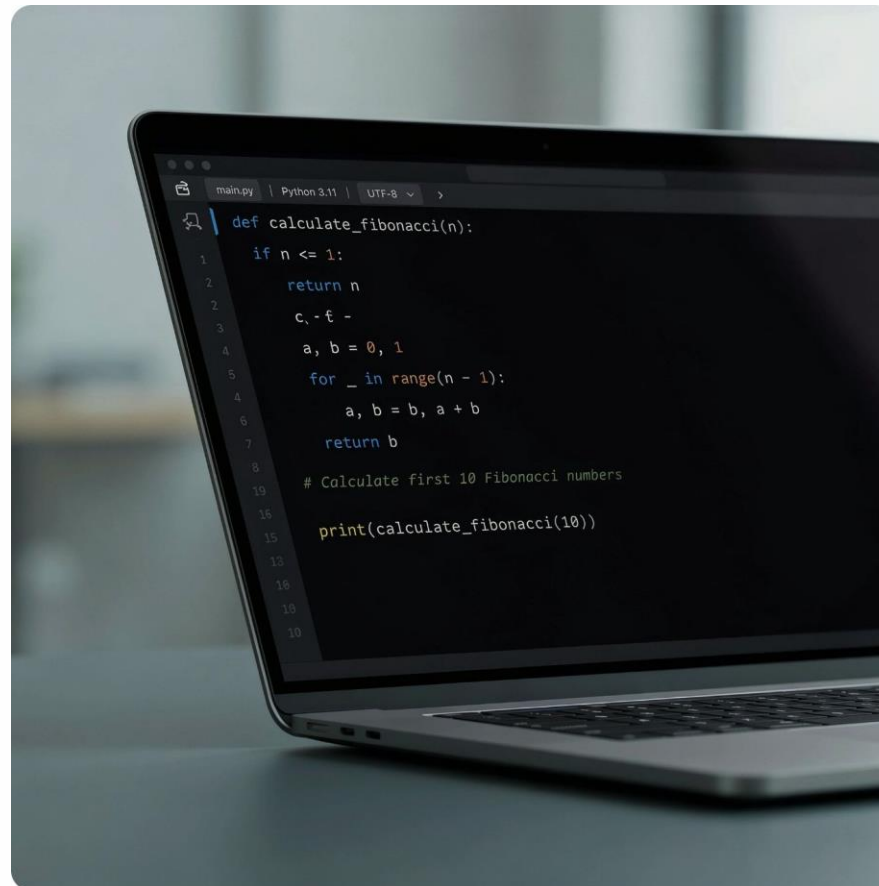
```
def say_hello():  
    print("안녕하세요")  
  
say_hello()
```

## 용어

- def: define의 약자로 함수 정의를 시작하는 키워드
- 함수 정의(Function Definition): 함수를 만드는 것
- 함수 호출(Function Call): 만들어진 함수를 실행하는 것

## 주의

- 함수를 정의했다고 자동으로 실행되는 것은 아닙니다.
- 호출해야 함수 내부 코드가 실행됩니다.



## 함수 실행 흐름

### 코드

```
def say_hello():  
    print("2. 함수 실행")  
print("1. 함수 호출 전")  
say_hello()  
print("3. 함수 호출 후")
```



함수 호출

함수 내부 실행

호출 위치로 복귀

### 실행 순서

01

함수 정의를 기억합니다.

02

"1. 함수 호출 전" 출력

03

say\_hello() 호출

04

함수 내부 코드 실행

05

호출 위치로 돌아옴

06

"3. 함수 호출 후" 출력

함수 호출 시 실행 흐름이 함수 내부로 이동했다가 다시 돌아옵니다.

# 매개변수(Parameter)와 인수(Argument)

## 예제

```
def greet(name):  
    print(f"{name}님, 안녕하세요")  
greet("민수")
```

## 매개변수(Parameter)

함수 정의 부분에서 입력값을 받을 변수

위 예제의 `name`

## 인수(Argument)

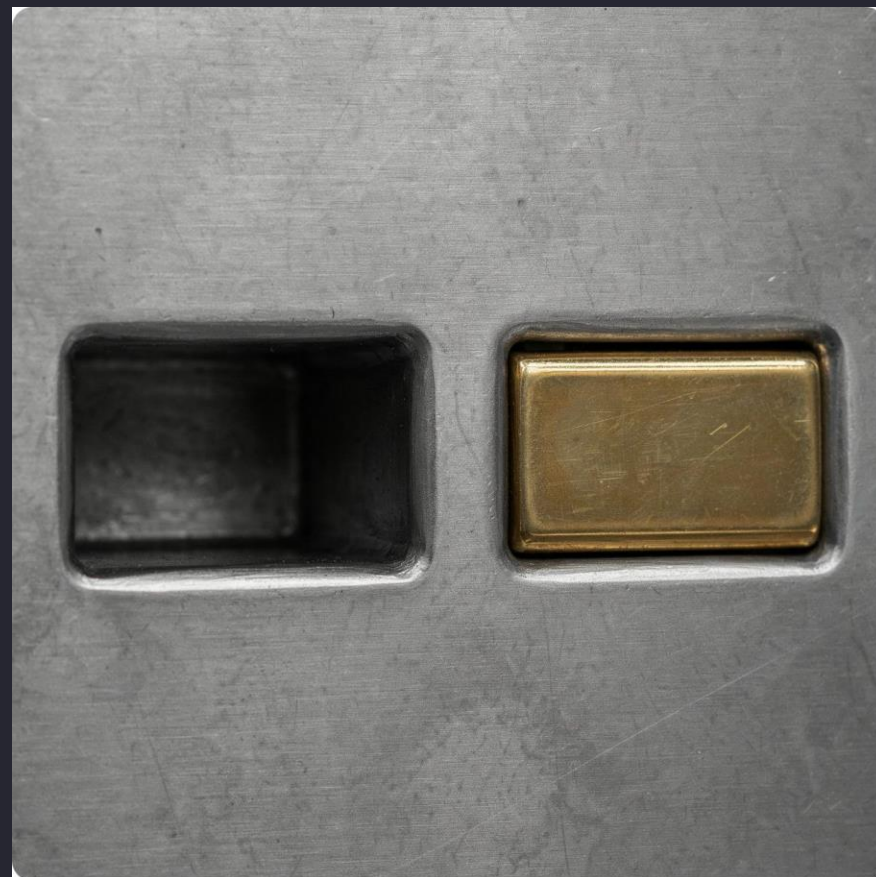
함수를 호출할 때 실제로 전달하는 값

위 예제의 `"민수"`

## 기억법

Parameter = 받을 자리

Argument = 실제로 넣는 값



# 여러 개의 매개변수

## 예제

```
def add(a, b):  
    result = a + b  
    return result  
print(add(10, 20))
```

## 설명

- a, b: 매개변수 2개
- 10, 20: 인수 2개
- 함수는 입력값을 여러 개 받을 수 있습니다.

## 의사코드

함수 add(첫번째수, 두번째수)  
합계 = 첫번째수 + 두번째수  
합계를 반환한다

## 출력

30

# 위치 인수(Positional Argument)

## 예제

```
def introduce(name, job):  
    print(f"이름: {name}, 희망직무: {job}")  
introduce("지민", "AI 개발자")
```

## 잘못된 순서 예

```
introduce("AI 개발자", "지민")
```

## 특징

- 전달하는 위치에 따라 매개변수와 값이 연결됩니다.
- 첫 번째 값 → 첫 번째 매개변수
- 두 번째 값 → 두 번째 매개변수

## 결과

문법 오류는 없지만 의미가 뒤바뀝니다.

⚠ 인수의 순서가 매개변수의 순서와 일치해야 합니다.

# 키워드 인수(Keyword Argument)

## 예제

```
def introduce(name, job):  
    print(f"이름: {name}, 희망직무: {job}")  
introduce(job="데이터 분석가", name="서연")
```

## 특징

- 매개변수 이름을 지정해서 값을 전달합니다.
- 호출 순서가 달라도 의미가 명확합니다.

## 장점

- 인수가 많을 때 가독성이 좋아집니다.
- 어떤 값이 어떤 역할인지 쉽게 알 수 있습니다.

# 기본값 매개변수(Default Parameter)

## 예제

```
def greet(name, message="안녕하세요"):
    print(f"{name}님, {message}")
greet("민수")
greet("지수", "반갑습니다")
```

## 활용 사례

출력 형식의 기본값

할인율의 기본값

언어 설정의 기본값

## 설명

- `message`에 기본값 "안녕하세요"를 지정했습니다.
- 인수를 생략하면 기본값을 사용합니다.
- 필요하면 다른 값을 전달할 수 있습니다.

# return: 함수의 결과를 돌려주기

## 예제

```
def add(a, b):  
    return a + b  
result = add(10, 20)  
print(result)
```

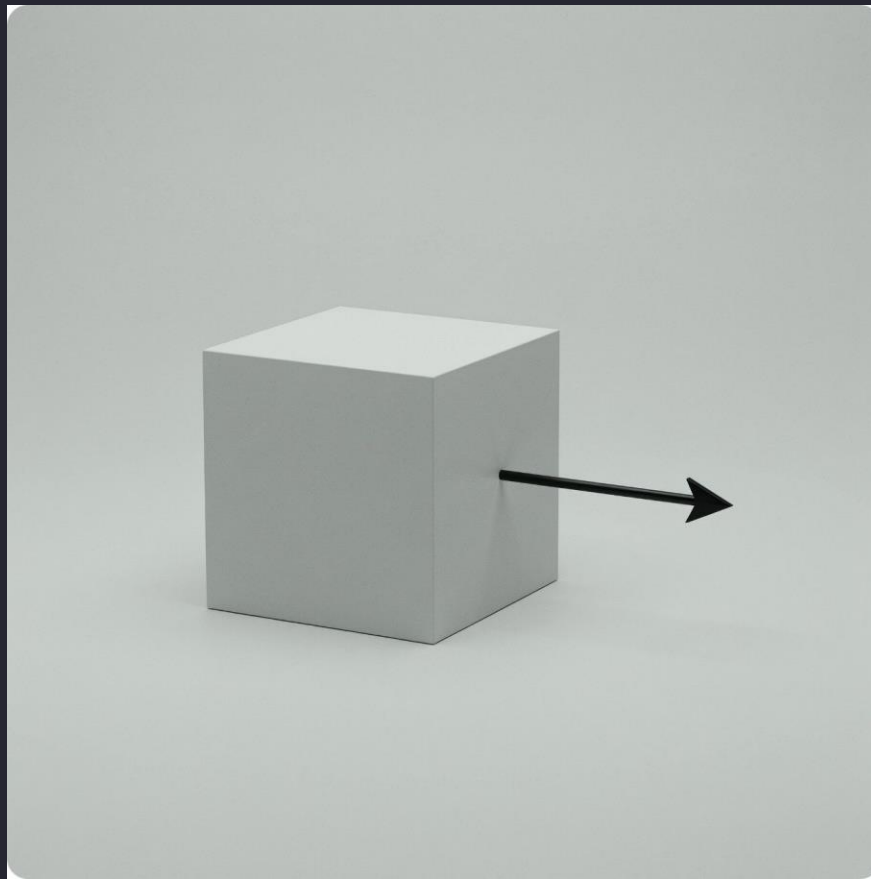
## return의 역할

- 함수 내부의 처리 결과를 함수 밖으로 전달합니다.
- return이 실행되면 함수 실행이 종료됩니다.

## 반환값(Return Value)

함수가 호출한 곳으로 돌려주는 값

❑ 계산을 담당하는 함수는 print()보다 return을 활용하면 재사용성이 높아집니다.



# print()와 return의 차이

## print()

- 화면에 값을 보여주는 함수
- 출력 자체가 목적

## return

- 함수의 결과값을 호출한 곳으로 전달
- 다른 계산에 다시 사용할 수 있음

## 비교

```
def add_print(a, b):  
    print(a + b)  
def add_return(a, b):  
    return a + b
```

## 활용

```
value = add_return(10, 20)  
print(value * 2)
```

## 핵심 질문

결과를 나중에 다시 사용해야 하는가?

그렇다면 return을 사용합니다.

# return을 만나면 함수는 종료됩니다

## 예제

```
def check_score(score):  
    if score >= 80:  
        return "PASS"  
    return "RETRY"
```

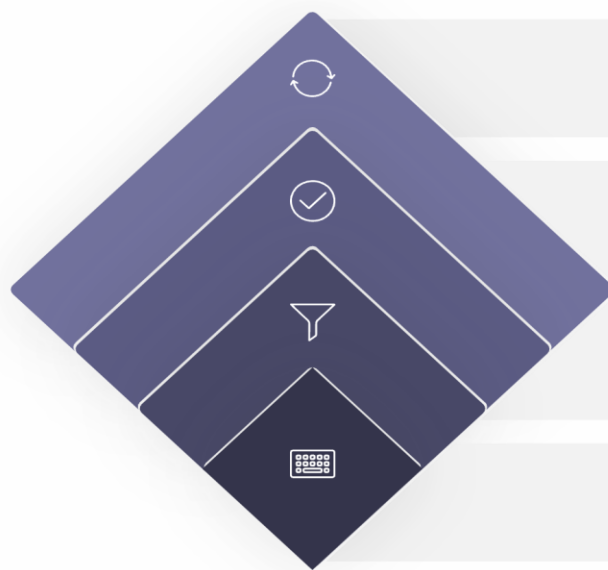
## 동작

- score가 80 이상이면 "PASS"를 반환하고 즉시 종료됩니다.
- 그렇지 않으면 아래의 "RETRY"를 반환합니다.

## 주의 예제

```
def test():  
    return 10  
    print("이 코드는 실행되지 않습니다")
```

⚠ return 아래의 코드는 실행되지 않습니다.



### RETRY 반환

그렇지 않으면 종료

### PASS 반환

80 이상인 경우 즉시 종료

### 조건 검사

score >= 80 인가?

### 입력 점수

score 값을 확인

# 여러 값을 반환하는 방법

## 예제

```
def get_result(kor, eng):  
    total = kor + eng  
    average = total / 2  
    return total, average  
sum_score, avg_score = get_result(90, 80)  
print(sum_score, avg_score)
```

## 설명

- return 뒤에 여러 값을 쉼표로 구분할 수 있습니다.
- 호출 시 여러 변수로 나누어 받을 수 있습니다.

## 용어

Packing: 여러 값을 하나로 묶음

Unpacking: 묶인 값을 여러 변수에 나누어 저장

# 변수의 범위(Scope)

## Scope

변수를 사용할 수 있는 유효 범위입니다.

## 지역변수(Local Variable)

- 함수 내부에서 만들어진 변수
- 일반적으로 함수 내부에서만 사용 가능

## 전역변수(Global Variable)

- 함수 밖에서 만들어진 변수
- 여러 위치에서 참조 가능

## 예제

```
course = "AI휴먼" # 전역변수
def show():
    day = 6 # 지역변수
    print(course, day)
```



# 지역변수(Local Variable)

## 예제

```
def calculate():  
    result = 100  
    print(result)  
calculate()
```

result는 calculate() 내부에서 만든 지역변수입니다.

## 잘못된 예

```
print(result)
```

## 예상 오류

⊗ NameError

## 이유

함수 밖에서는 지역변수 result를 찾을 수 없습니다.

# 전역변수와 global 키워드

## 예제

```
count = 0
def increase():
    global count
    count += 1
```

## 설명

함수 내부에서 전역변수 값을 변경하려면 `global` 키워드를 사용할 수 있습니다.

## 하지만 권장 원칙

- 가능한 한 전역변수 변경을 줄입니다.
- 입력은 매개변수로 받고 결과는 `return`으로 반환하는 구조가 이해하기 쉽습니다.

## 더 좋은 구조

```
def increase(count):
    return count + 1
```

# 함수 + 조건문: 성적 등급 판정

## 예제

```
def get_grade(score):  
    if score >= 90:  
        return "A"  
    elif score >= 80:  
        return "B"  
    elif score >= 70:  
        return "C"  
    elif score >= 60:  
        return "D"  
    else:  
        return "F"
```

## 호출

```
print(get_grade(87))
```

## 출력

B

90점 이상

A

80점 이상

B

70점 이상

C

60점 이상

D

60점 미만

F

함수 이름만 봐도 역할을 이해할 수 있도록 작성합니다.

# 함수 + 리스트: 평균 계산

## 예제

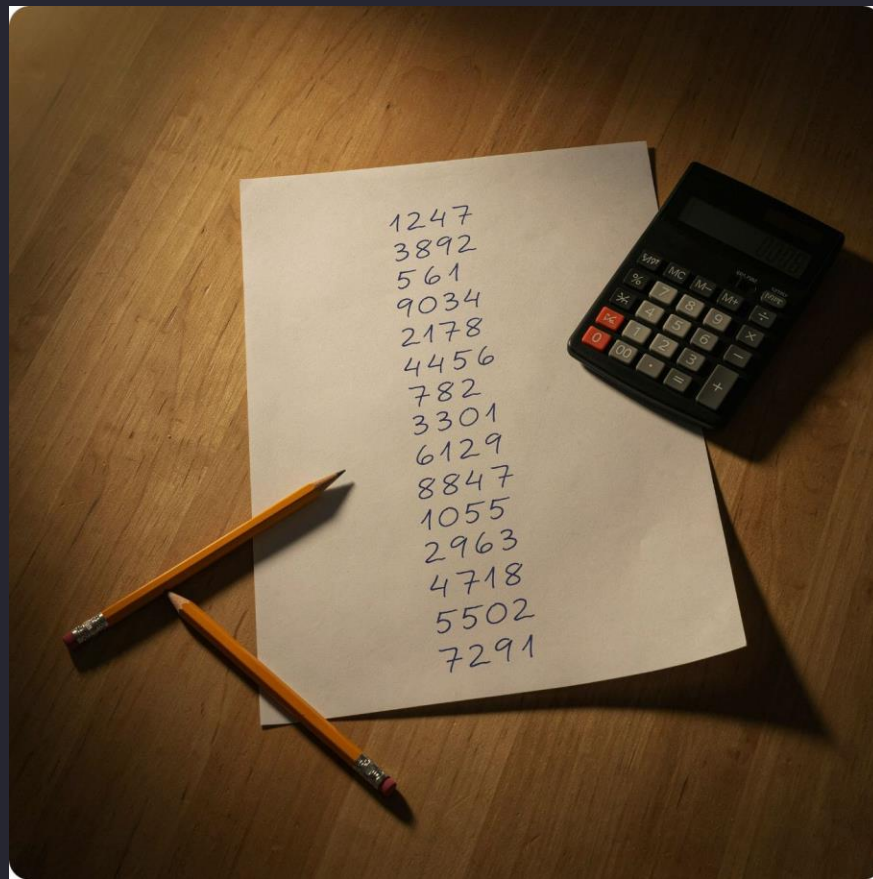
```
def get_average(scores):  
    total = 0  
    for score in scores:  
        total += score  
    return total / len(scores)
```

## 호출

```
scores = [80, 90, 70]  
print(get_average(scores))
```

## 함수의 장점

어떤 점수 리스트가 와도 같은 평균 계산 로직을 재사용할 수 있습니다.



# 함수 + 딕셔너리: 학생 데이터 처리

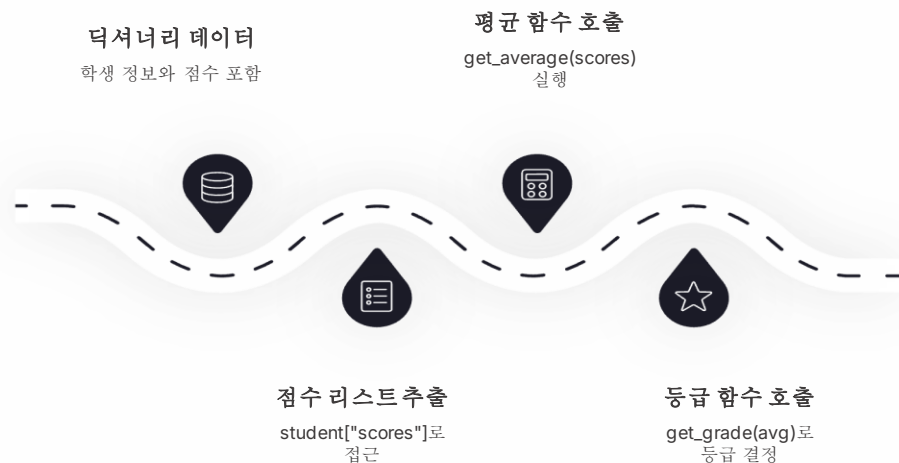
## 데이터

```
student = {  
    "name": "김민수",  
    "scores": [85, 90, 78]  
}
```

## 함수 호출

```
avg = get_average(student["scores"])  
grade = get_grade(avg)
```

## 처리 흐름



# 큰 프로그램을 함수로 나누기

한 함수에 모든 코드를 넣지 않습니다.

## 역할 분리 예



`get_average(scores)`

평균 계산



`get_grade(score)`

등급 판정



`print_student_report(student)`

학생 정보 출력

## 장점

- 각 함수의 역할이 명확합니다.
- 일부 기능만 수정하기 쉽습니다.
- 함수별로 테스트하기 쉽습니다.

□ 원칙: 한 함수 = 하나의 명확한 역할

# 대표 실습: 함수형 성적 처리 프로그램

## 요구사항

- 학생 이름과 점수 목록을 딕셔너리로 관리
- 평균을 구하는 함수 작성
- 평균 점수에 따라 등급을 반환하는 함수 작성
- 학생별 결과를 출력하는 함수 작성

## 예상 출력

김민수 | 평균: 84.3 | 등급: B  
이서연 | 평균: 93.0 | 등급: A  
박준호 | 평균: 68.7 | 등급: D

## 학습 포인트

함수 입력

함수 반환값

함수 간 호출

리스트 반복 처리

딕셔너리 활용

# 대표 실습 의사코드(Pseudocode)

## 의사코드

함수 평균계산(점수목록)

합계를 0으로 설정한다

점수목록을 반복한다

합계에 점수를 더한다

평균 = 합계 / 점수개수

평균을 반환한다

함수 등급판정(평균)

평균이 90 이상이면 A 반환

평균이 80 이상이면 B 반환

평균이 70 이상이면 C 반환

평균이 60 이상이면 D 반환

그 외에는 F 반환

함수 결과출력(학생)

학생의 점수목록으로 평균계산 함수 호출

평균값으로 등급판정 함수 호출

이름, 평균, 등급 출력

학생목록을 반복한다

결과출력 함수를 호출한다



평균계산

등급판정

결과출력

# 자주 발생하는 오류

## 함수 호출 시 괄호 누락

```
print(get_grade)  # 함수 자체를  
                  가리킴  
print(get_grade(85))  # 함수 호출  
                    결과
```

## 인수 개수 불일치

```
def add(a, b):  
    return a + b  
add(10)  # TypeError
```

## return 결과를 사용하지 않음

```
get_grade(85)  # 반환값은  
              있지만 저장·출력하지 않음
```

## 지역 변수를 함수 밖에서 사용

NameError 발생 가능

## 함수 안에서 print만 하고 계산 결과를 재사용하려고 함

계산 함수는 return을 우선 고려

# 디버깅 체크리스트

함수가 예상대로 동작하지 않을 때 확인

- 1 함수가 실제로 호출되었는가?
- 2 전달한 인수의 개수와 순서가 맞는가?
- 3 인수의 자료형이 예상과 같은가?
- 4 return 문이 필요한 위치에 있는가?
- 5 return 이후에 실행하려는 코드는 없는가?
- 6 지역변수를 함수 밖에서 사용하고 있지 않은가?
- 7 함수 이름과 변수 이름이 충돌하지 않는가?
- 8 작은 입력값으로 함수 하나씩 테스트했는가?

# 오늘의 과제 구성

## Basic

- 두 수를 받아 합계를 반환하는 함수 작성
- 함수 정의, 매개변수, `return`에 집중

## Application

- 성적 등급 판정 로직을 함수로 분리
- 여러 학생의 점수를 처리

## Challenge

- 평균 계산, 등급 판정, 결과 출력을 각각 함수로 분리
- 함수 간 호출 구조를 설계

# 과제 진행 원칙

01

먼저 의사코드를 작성합니다.

02

함수 하나를 작성한 뒤 바로 테스트합니다.

03

동작 확인 후 다음 함수를 추가합니다.

# 핵심 용어 정리

- 1 Function(함수)**  
특정 작업을 수행하도록 묶어 놓은 코드 블록
- 2 Parameter(매개변수)**  
함수 정의 시 입력값을 받을 변수
- 3 Argument(인수)**  
함수 호출 시 실제로 전달하는 값
- 4 Return Value(반환값)**  
함수가 호출한 곳으로 돌려주는 결과
- 5 Scope(변수의 범위)**  
변수를 사용할 수 있는 유효 범위
- 6 Local Variable(지역변수)**  
함수 내부에서 생성되어 주로 함수 내부에서 사용하는 변수
- 7 Global Variable(전역변수)**  
함수 밖에서 생성되어 넓은 범위에서 참조 가능한 변수

# 오늘의 정리와 다음 학습 연결

## 오늘의 핵심

- 반복 코드를 함수로 묶으면 재사용할 수 있습니다.
- 함수는 매개변수로 입력을 받고 `return`으로 결과를 전달할 수 있습니다.
- 지역변수와 전역변수는 사용할 수 있는 범위가 다릅니다.
- 큰 프로그램은 작은 함수로 나누면 이해와 수정이 쉬워집니다.

## 오늘의 결과물

함수형 성적 처리 프로그램

## 다음 학습 연결

- 파일 입출력(File I/O)
- 예외 처리(Exception Handling)
- 모듈(Module)과 `import`

다음에는 함수로 만든 로직을 파일 데이터와 연결하고, 오류 상황에도 안전하게 실행되는 프로그램으로 확장합니다.

